Instrucciones para la Actividad 3: Investigación sobre GitHub y Trabajo en Ramas

En esta actividad, realizarás una investigación sobre el uso de GitHub y el trabajo en ramas. El objetivo es que entiendas los conceptos fundamentales y cómo aplicarlos en la gestión de proyectos de desarrollo de software. A continuación, te proporcionamos 10 preguntas que deberás responder basándote en tus lecturas, videos, o recursos disponibles en línea.

Instrucciones:

1. Investiga los conceptos clave relacionados con GitHub y el trabajo en ramas.
2. Responde las 10 preguntas proporcionadas de manera clara y concisa, utilizando ejemplos si es necesario.
3. Las respuestas deberán entregarse en un documento, ya sea en formato PDF o Word, y subirse a la plataforma indicada por el instructor.
4. Sé detallado en tus respuestas, asegurándote de que sean comprensibles para alguien que no está familiarizado con GitHub o el trabajo en ramas.

Recursos sugeridos para la investigación:

- Documentación oficial de Git: https://git-scm.com/doc
- Documentación oficial de GitHub: https://docs.github.com
- Videos tutoriales de GitHub en YouTube
- Artículos en blogs o tutoriales sobre Git y GitHub

Preguntas sobre GitHub y Trabajo en Ramas:

1. **¿Qué es Git y cuál es su relación con GitHub?**

Git es un sistema de control de versiones que se instala en la computadora para registrar los cambios en los archivos de un proyecto a lo largo del tiempo. GitHub es una plataforma en la nube que aloja repositorios de Git. Se complementan porque Git gestiona el código de forma local y GitHub permite subirlo a internet para compartirlo, respaldarlo y trabajar en equipo.

2. **¿Qué es una rama (branch) en Git y por qué es importante en el control de versiones?**

Una rama es una línea de desarrollo independiente dentro del mismo proyecto, como un borrador o un camino paralelo. Es importante porque permite a los desarrolladores trabajar en nuevas funciones o reparar fallas sin alterar la versión principal del código que está funcionando. En el trabajo colaborativo evita que los cambios de una persona afecten el trabajo de los demás antes de tiempo.

3. **¿Cómo se crea una nueva rama en Git? Escribe el comando y explica su uso.**

Se crea con el comando git branch nombre-de-la-rama. Se debe crear una rama cada vez que se vaya a trabajar en una nueva tarea, función o corrección específica para mantener el proyecto organizado.

4. **¿Cuál es la diferencia entre git merge y git rebase?**

git merge une dos ramas añadiendo un nuevo commit de fusión, conservando todo el historial original de ambas ramas intacto. git rebase toma los cambios de una rama y los aplica encima de otra, reescribiendo la historia para que parezca una secuencia en línea recta. Es preferible usar git merge cuando se quiere conservar el historial real de colaboración y git rebase cuando se busca un historial más limpio.

5. **¿Qué es el comando git checkout y cómo se utiliza para cambiar entre ramas?**

git checkout nombre-de-la-rama sirve para navegar y posicionarse en la rama deseada, actualizando los archivos de la computadora para mostrar el estado exacto de esa rama. Un comando alternativo más moderno para cambiar de rama es git switch nombre-de-la-rama.

6. **¿Cómo puedes visualizar todas las ramas disponibles en un repositorio de Git?**

Se utiliza el comando git branch (o git branch -a para incluir las remotas). Su salida muestra una lista con los nombres de todas las ramas existentes, marcando con un asterisco y un color diferente la rama en la que te encuentras parado actualmente.

7. **¿Qué es un Pull Request (PR) en GitHub y cómo se utiliza en el flujo de trabajo colaborativo?**

Un Pull Request es una petición formal en la plataforma de GitHub donde un desarrollador propone integrar los cambios de su rama hacia la rama principal. Es clave porque permite al equipo revisar el código nuevo, comentar correcciones y probarlo antes de autorizar su unión al proyecto.

8. **¿Qué sucede cuando fusionas (merge) una rama en la rama principal (main o master)?**

Al fusionar, Git toma todos los commits realizados en la rama secundaria y los integra en la rama principal. El código de la rama principal se actualiza incorporando las novedades sin perder su historial previo, unificando el trabajo de ambas partes.

9. **¿Cómo puedes resolver un conflicto de fusión (merge conflict) en Git?**

Un conflicto ocurre cuando dos personas modifican la misma línea de un mismo archivo en diferentes ramas y Git no sabe cuál versión conservar. Para resolverlo, se abre el archivo en conflicto, se revisan las marcas generadas por Git (<<<<<<<, =======, >>>>>>>), se borra manualmente el código sobrante dejando la versión correcta, se guardan los cambios, y se concluye la fusión haciendo un git add seguido de un git commit.

10. **¿Qué es el comando `git branch -d` y cuándo se debe utilizar?**

El comando git branch -d nombre-de-la-rama sirve para eliminar una rama localmente. Debe utilizarse únicamente cuando la tarea de esa rama ya fue terminada y sus cambios ya fueron fusionados con éxito en la rama principal, evitando guardar código duplicado o innecesario.

Integrantes:
Cristian Avendaño del Solar
Osler René Catillo Estrada
Alondra Iridany Saucedo Jiménez
Alejandro Candia Hernández
Angel Emmanuel Leyva Muñiz
Brandon Arreola Cortés